

C# Interview Questions & Answers

50 Questions Covering Core Concepts, OOP, and Language Fundamentals

C# Basics

1. What are the main features of C# as a language?

C# is a modern, object-oriented, type-safe language that runs on the .NET runtime (CLR). Key features include automatic memory management via garbage collection, strong static typing with type inference, support for both structured and functional programming (LINQ, lambdas), component-oriented design (properties, events, assemblies), and built-in support for asynchronous programming through `async/await`.

2. What is the difference between value types and reference types?

Value types (`struct`, `int`, `bool`, `enum`, etc.) store their data directly in the memory location where the variable is declared — typically the stack. Reference types (`class`, `string`, `array`, `delegate`) store a reference (pointer) to the data, which lives on the heap. Copying a value type copies the actual data; copying a reference type copies the reference, so both variables point to the same object.

3. What are the built-in data types in C#, and how do `int`, `long`, `float`, `double`, and `decimal` differ?

C# has built-in value types like `int` (32-bit integer), `long` (64-bit integer), `float` (32-bit floating point), `double` (64-bit floating point), and `decimal` (128-bit, high-precision, used for financial calculations). `float` and `double` trade precision for speed and can introduce rounding errors, while `decimal` is exact for base-10 arithmetic, making it the right choice for currency.

4. What is the difference between `var`, `dynamic`, and explicitly typed variables?

`var` lets the compiler infer the type at compile time based on the assigned value — the type is still static and fixed after declaration. `dynamic` bypasses compile-time type checking entirely; the type is resolved at runtime, which offers flexibility but loses compile-time safety and IntelliSense. Explicitly typed variables declare the type directly and are the most restrictive but clearest option.

5. What is type casting, and what's the difference between implicit and explicit casting?

Type casting converts a value from one type to another. Implicit casting happens automatically when there's no risk of data loss (e.g., `int` to `long`). Explicit casting requires a cast operator because the conversion may lose data or fail (e.g., `double` to `int`, or `object` to a specific class), and the compiler forces

you to acknowledge that risk explicitly.

6. What is the difference between `const` and `readonly`?

`const` fields are evaluated at compile time, must be assigned at declaration, and are implicitly static.

`readonly` fields are evaluated at runtime and can be assigned either at declaration or inside a constructor, allowing different instances to have different `readonly` values.

Classes & Objects

7. What is the difference between a class and an object?

A class is a blueprint or template that defines the structure (fields, properties) and behavior (methods) of a type. An object is a concrete instance of that class, created using the `new` keyword, with its own state in memory.

8. What are static classes used for, and can they be instantiated?

A static class cannot be instantiated — it exists purely to group related static members (fields, methods) that don't need per-instance state, such as utility or helper classes (e.g., `Math`, `Console`). All members must be static, and the class is implicitly sealed.

Constructors

9. What is a constructor, and what types exist?

A constructor is a special method invoked when an object is created, used to initialize its state. Types include: default constructor (no parameters, auto-generated if none defined), parameterized constructor (accepts arguments), copy constructor (initializes a new object from an existing one, not built-in but commonly hand-written), static constructor (initializes static members, runs once per type), and private constructor (restricts instantiation, often used in singleton patterns).

10. What is constructor chaining, and how do you implement it using `this()` and `base()`?

Constructor chaining lets one constructor call another to avoid duplicating initialization logic. Use `this(...)` to call another constructor in the same class, and `base(...)` to call a constructor in the parent class. `base()` is called implicitly if not specified explicitly.

```
public class Employee
{
```

```
public Employee() : this("Unknown") { }  
public Employee(string name) { Name = name; }  
public string Name;  
}
```

Encapsulation

11. What is encapsulation, and how do access modifiers support it?

Encapsulation is the practice of bundling data and the methods that operate on it within a class, while restricting direct access to internal state. Access modifiers control visibility: private (class only), protected (class and derived classes), internal (same assembly), protected internal (derived classes or same assembly), private protected (derived classes within the same assembly only), and public (unrestricted).

12. Why prefer properties over public fields?

Properties let you add validation, computed logic, or change notification without breaking the public API, and you can make the getter and setter have different access levels. Public fields expose raw memory directly, so if you later need to add logic (like validation), you'd have to change the public interface, breaking any code that depends on it.

Abstraction

13. What is abstraction, and how do abstract classes achieve it?

Abstraction means exposing only essential behavior while hiding implementation details. An abstract class can declare abstract methods (no body, must be implemented by derived classes) alongside concrete methods, letting consumers work against a high-level contract without knowing the underlying implementation.

14. How does abstraction differ from encapsulation in practice?

Abstraction is about hiding complexity and exposing only what's necessary at the design level (what an object does). Encapsulation is about hiding data and controlling access at the implementation level (how an object protects its state). Abstraction is achieved via interfaces and abstract classes; encapsulation is achieved via access modifiers and properties.

Inheritance

15. What is inheritance, and what types does C# support? Why no multiple class inheritance?

Inheritance allows a class to acquire the members of another class. C# supports single, multilevel, and hierarchical inheritance for classes. It does not support multiple inheritance of classes to avoid the "diamond problem" (ambiguity when two base classes define the same member). Multiple inheritance of behavior is instead achieved through interfaces.

16. What is the base keyword used for, and how does it differ from this?

base refers to the immediate parent class, used to call a base constructor, method, or property that's been overridden. this refers to the current instance of the class itself. base is about reaching upward in the hierarchy; this is about referring to the current object.

17. Can a derived class access private members of its base class?

No. Private members are accessible only within the class where they're declared. A derived class can access protected or public members of its base class, but not private ones — even though private fields still exist in memory for the derived object, they can't be referenced directly.

Interfaces

18. What is an interface, and how does it differ from an abstract class?

An interface defines a contract — a set of members a class must implement — with historically no implementation or state. An abstract class can have both abstract and concrete members, fields, and constructors. A class can implement multiple interfaces but inherit from only one abstract class, since C# doesn't support multiple class inheritance.

19. Can an interface have a method implementation (default interface methods, C# 8+)?

Yes, since C# 8.0, interfaces can include default implementations for methods. This allows adding new members to an interface without breaking existing implementing classes, since they inherit the default behavior unless they choose to override it.

20. Can a class implement multiple interfaces? How are naming conflicts resolved?

Yes, a class can implement any number of interfaces. If two interfaces declare a member with the same name, you resolve the conflict using explicit interface implementation, which lets you provide separate implementations accessible only when the object is cast to the specific interface.

```
void IFirst.Show() { /* ... */ }  
void ISecond.Show() { /* ... */ }
```

Polymorphism

21. What is polymorphism, and what's the difference between compile-time and run-time polymorphism?

Polymorphism allows objects of different types to be treated through a common interface. Compile-time polymorphism is achieved via method overloading and operator overloading, resolved at compile time based on signatures. Run-time polymorphism is achieved via method overriding with virtual/override, resolved at runtime based on the actual object type.

22. What's the difference between virtual/override and method hiding with new?

virtual/override participates in run-time polymorphism — calling the method through a base class reference invokes the derived class's overridden version. new hides the base method instead of overriding it; calling through a base class reference invokes the base version, while calling through a derived reference invokes the new one, breaking polymorphic behavior.

23. Can you override a non-virtual method?

No. A method must be marked virtual (or abstract) in the base class before it can be overridden. Attempting to use override on a non-virtual method causes a compile-time error; you could only hide it using the new keyword instead.

Aggregation & Composition

24. What is aggregation, and how is a has-a relationship represented in C#?

Aggregation models a "has-a" relationship where one class holds a reference to another, but the contained object can exist independently of the container. It's represented simply by one class holding a field or property of another class's type, often passed in via constructor or property, without owning its lifecycle.

25. What's the difference between aggregation and composition?

In aggregation, the contained object's lifetime is independent of the container (e.g., a University has Students, but Students can exist without the University). In composition, the contained object's lifetime is tied to the container (e.g., a House has Rooms; the Rooms cease to exist if the House is destroyed). Composition implies stronger ownership.

26. Why is composition often preferred over inheritance?

Composition provides more flexibility because behavior can be composed and swapped at runtime, whereas inheritance creates a rigid, compile-time-fixed hierarchy. It also avoids issues like fragile base class problems, where changes to a base class ripple unexpectedly into derived classes.

Method Overloading & Overriding

27. What is method overloading, and what rules govern it?

Overloading lets multiple methods share the same name but differ in parameter count, order, or type. Return type alone cannot distinguish overloads — the parameter signature must differ. The compiler resolves which overload to call based on the arguments passed at compile time.

28. What is method overriding, and what conditions must be met?

Overriding lets a derived class provide a new implementation for a method inherited from its base class. The base method must be marked virtual, abstract, or override, and the derived method must use the override keyword with a matching signature and accessibility.

Sealed Classes & Members

29. What is a sealed class, and why would you use one?

A sealed class cannot be inherited from. You'd seal a class to prevent further derivation — for performance reasons (the JIT compiler can make certain optimizations), for security (preventing malicious subclassing), or simply because the design doesn't intend the class to be extended.

30. Can you seal a single overridden method without sealing the whole class?

Yes. You can mark an overridden method as sealed override in a derived class. This prevents any further derived classes from overriding that specific method again, while the class itself remains inheritable.

Structs vs Classes

31. What is the core difference between a struct and a class?

A struct is a value type, allocated on the stack (when it's a local variable) and copied by value, while a class is a reference type, allocated on the heap and copied by reference. Structs are best for small,

immutable data (like a Point or Color); classes are better for larger objects or ones that need identity and mutability across references.

32. Why can't structs support inheritance?

Structs are sealed by default and cannot inherit from another struct or class (though they can implement interfaces). This is a deliberate design choice tied to their value-type semantics — supporting inheritance would complicate their fixed-size, stack-friendly memory layout.

Delegates & Events

33. What is a delegate, and how does it work?

A delegate is a type-safe function pointer — it holds a reference to a method with a matching signature and can invoke that method indirectly. Delegates enable passing methods as parameters, implementing callbacks, and are the foundation for events and LINQ's functional-style APIs.

```
public delegate int Operation(int a, int b);  
Operation add = (x, y) => x + y;  
int result = add(2, 3);
```

34. What's the difference between a delegate and an event?

An event is a wrapper around a delegate that restricts how it can be used from outside the declaring class: external code can only subscribe (+) or unsubscribe (-), not invoke the event directly or overwrite the entire invocation list. This enforces the publisher-subscriber pattern and protects encapsulation.

35. What is a multicast delegate?

A multicast delegate holds references to more than one method, all with the same signature, and invokes them in sequence when called. You build one by combining delegates with the + or += operator; if any method has a return value, only the last invoked method's result is returned.

Generics

36. What are generics, and what problem do they solve?

Generics let you define classes, methods, and interfaces with a placeholder type parameter (like List<T>), specified when the type is used. They solve the problem of writing type-safe, reusable code without duplicating logic per type and without the performance and safety costs of casting to/from object, which was the pre-generics approach.

37. What are generic constraints, and why use them?

Constraints restrict what types can be used as a type parameter, using the where clause (e.g., where T : class, where T : new(), where T : IComparable). They're used to guarantee the generic code can safely call specific members or operations on T, such as requiring a parameterless constructor or a particular interface.

```
public class Repository<T> where T : class, new()
{
    public T CreateNew() => new T();
}
```

Exception Handling

38. What is the exception hierarchy in C#?

All exceptions derive from System.Exception. Below it, SystemException covers exceptions thrown by the CLR itself (like NullReferenceException, IndexOutOfRangeException), while ApplicationException was originally intended for user-defined exceptions, though modern guidance recommends deriving custom exceptions directly from Exception instead.

39. What's the difference between throw and throw ex in a catch block?

throw re-throws the current exception while preserving the original stack trace, making it easier to diagnose where the error actually occurred. throw ex re-throws the exception but resets the stack trace to the current catch block, losing the original point of failure — generally considered bad practice.

Properties & Indexers

40. What is a property, and how does it differ from a public field at the IL level?

A property looks like a field syntactically but compiles down to a pair of get_ and set_ methods under the hood. This means properties can include logic (validation, lazy loading) and can evolve without breaking binary compatibility, whereas a public field is accessed directly and offers no such flexibility.

41. What is an indexer, and when would you use one?

An indexer lets an object be accessed using array-like syntax (obj[index]) by defining a special this[] property. It's useful for custom collection or wrapper classes where you want consumers to retrieve or set elements naturally, without exposing the underlying storage directly.

```
public class Matrix
{
    private double[,] data = new double[10, 10];
    public double this[int row, int col]
    {
        get => data[row, col];
        set => data[row, col] = value;
    }
}
```

Static Members

42. What's the difference between an instance member and a static member?

An instance member belongs to a specific object and each instance has its own copy of the data. A static member belongs to the type itself and is shared across all instances — there's only one copy in memory regardless of how many objects are created.

43. Can a static method access instance members? Why or why not?

No, not directly. A static method has no implicit reference to any particular object (there's no 'this'), so it cannot access instance fields, properties, or methods unless it's given a specific instance to work with (e.g., as a parameter).

Object Class & Overrides

44. Why must Equals() and GetHashCode() be overridden together?

Types that are used as keys in hash-based collections (like Dictionary or HashSet) rely on the contract that if two objects are Equal, they must produce the same hash code. If you override Equals() without GetHashCode(), equal objects could end up in different hash buckets, breaking lookups and causing subtle bugs.

45. What's the difference between overriding ToString() and the default behavior?

The default object.ToString() returns the fully qualified type name (e.g., MyNamespace.MyClass). Overriding ToString() lets you return a meaningful, human-readable string representation of the object's state, which is especially useful for logging, debugging, and UI display.

Nested & Partial Classes

46. What is a nested class, and when might you use one?

A nested class is a class defined inside another class. It's useful when the nested type is tightly coupled to its enclosing class and has no meaning outside that context — for example, a private helper class used only internally, or an Enumerator implementation for a custom collection.

47. What is a partial class, and what's a common use case?

A partial class lets a single class's definition be split across multiple files using the `partial` keyword; the compiler merges them at build time. A common use case is separating auto-generated code (like designer files in WinForms, or Entity Framework scaffolded models) from hand-written custom logic, so regeneration doesn't overwrite manual edits.

Collections & Arrays

48. What is the difference between an array and a collection like List in C#?

An array has a fixed size determined at creation and cannot grow or shrink. `List<T>` is a dynamic, resizable collection built on top of an array internally, which automatically handles resizing as elements are added or removed, at the cost of some overhead compared to a raw array.

49. What is the difference between IEnumerable and ICollection?

`IEnumerable<T>` only supports forward-only iteration (via `GetEnumerator`) and knows nothing about size or mutability. `ICollection<T>` extends `IEnumerable<T>` and adds `Count`, `Add`, `Remove`, `Contains`, and `Clear`, representing an actual in-memory, modifiable collection rather than just a sequence to iterate.

Nullable Types

50. What is a nullable value type (int?), and how do ?? and ??= work with it?

A nullable value type wraps a value type (like `int`) in `Nullable<T>`, allowing it to also hold `null` — useful for representing "no value" in cases like optional database fields. The null-coalescing operator `??` returns the left-hand value if it's not null, otherwise the right-hand fallback. The null-coalescing assignment `??=` assigns the right-hand value only if the left-hand variable is currently null.

```
int? age = null;
int safeAge = age ?? 18; // safeAge = 18
age ??= 25; // age = 25 (was null)
```